Davante

2

PROGRAMACIÓN

Elementos de un programa informático

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Bloques de un programa 5

/ 3. Identificadores 6

/ 4. Caso práctico 1: “Variables y constantes” 7

/ 5. Tipos de datos: Primitivos 7

/ 6. Datos de tipo numérico 8

6.1. Tipo entero 9

6.2. Tipo real 9

/ 7. Datos de tipo alfanumérico 10

/ 8. Datos de tipo lógico 10

/ 9. Expresiones y operadores 11

9.1. Operadores aritméticos 12

9.2. Operadores lógicos 12

9.3. Operadores relacionales 13

9.4. Operadores opera y asigna 14

ÍNDICE

/ 10. Caso práctico 2: "Precedencia de operadores" 14

/ 11. Conversiones de tipo (casting) y comentarios 16

11.1. Castings implícitos (*Upper-casting*) 16

11.2. Castings explícitos (*Down-casting*) 16

/ 12. API de Java 17

12.1. Paquetes 17

12.2. Salida por consola 17

12.3. Entrada de datos 17

/ 13. Resumen y resolución del caso práctico de la unidad 18

/ 14. Bibliografía 19

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer qué es un bloque en programación.

Diferenciar correctamente entre identificador, variable y constante.

Distinguir los diferentes tipos de datos primitivos.

Saber cuándo utilizar cada tipo de datos.

Conocer y aplicar los diferentes operadores y expresiones.

Aplicar casting sobre diferentes tipos de datos.

Conocer y aplicar comentarios en código.

/ 1. Introducción y contextualización práctica

A la hora de empezar a programar es fundamental manejar los conceptos básicos, como son los bloques de programación, que permiten organizar el código y mantener un control.

La información que se maneja, es necesario almacenarla en espacios de memoria para poder realizar operaciones con ella. Esta información podrá ser de distintos tipos, por lo que se podrán realizar distintas operaciones sobre ella. Aunque existen operaciones especiales para convertir un tipo de dato en otro.

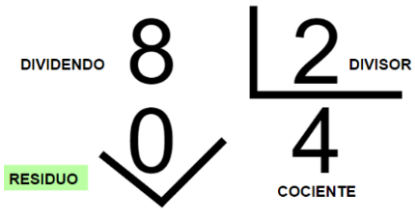


Fig.1. Número par.

Por último es necesario tener un código legible y comprensible para los distintos desarrolladores que lo manejen, así como documentar la funcionalidad y el comportamiento mediante el uso de comentarios.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "¿Par o impar?"
https://on.soundcloud.com/MR1qbupkjRmkWBvx9

/ 2. Bloques de un programa

Como ya se ha dicho anteriormente, cada lenguaje de programación dispone de una sintaxis y una semántica, reglas de programación e incluso tipos de datos representables, propios.

Por tanto, la mayoría de los **lenguajes de programación** cuentan con los siguientes **elementos comunes**:

- **Caracteres**: Son empleados para los datos e instrucciones.
- **Datos**: Son los objetos manipulables por el ordenador con un nombre (identificador) y un valor (contenido).
- **Identificadores**: Son los nombres que se utilizan para identificar componentes significativos de un programa.
- **Expresiones y operadores**: Mientras las expresiones son combinaciones de constantes, variables, símbolos de operación, paréntesis y nombres de funciones especiales, los **operadores** son símbolos que indican la operación a realizar.
- **Instrucciones**: Son expresiones que indican una operación a realizar por el ordenador.
- **Comentarios**: Son los mensajes a cualquier usuario que lea el código fuente cuya finalidad es explicarlo y hacerlo comprensible.

Sin embargo, al realizar un programa, indistintamente del lenguaje de programación que vayamos a emplear, hemos de seguir una estructura que viene ya definida por dicho lenguaje de programación.

Las estructuras vienen concretadas por lo que llamamos **bloques**. Un bloque nos va a indicar dónde podemos escribir código respetando la estructura del lenguaje de programación que utilicemos. Los bloques van a estar limitados por una serie de delimitadores, también especificados por el lenguaje de programación. Por ejemplo, en Java, los limitadores de bloques de código serán las llaves ({ }).

```
public class Ejemplo {
                                    Bloque de código 1
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here
    }   Bloque de código 2

}
```

Fig.2. Bloques de código.

Se pueden definir dos grandes bloques dentro de un programa informático: **bloque de declaración y bloque de instrucciones**.

El **bloque de declaración** es aquel en el que se definen todos los objetos que se utilizan a lo largo del programa, por ejemplo, constantes, variables, archivos, etc.

A su vez, el **bloque de instrucciones** define el conjunto de operaciones que se van a llevar a cabo sobre los datos de entrada para conseguir los resultados del procesamiento.

/ 3. Identificadores

Son un **conjunto de nombres que se utilizan para identificar componentes significativos de un programa**, como instrucciones, nombres de variables, constantes, etc. Estos identificadores no se podrán repetir, deberán empezar forzosamente por una letra, no tener espacios en blanco y ser significativos.

Distinguiremos **dos tipos de identificadores**:

- **Palabras reservadas**: Son identificadores predefinidos que tienen un **significado especial**. Son propios de cada lenguaje de programación. Por lo tanto, un identificador definido por el usuario no puede tener el mismo nombre que una palabra clave.

• abstract	• continue	• for	• new	• switch
• assert	• default	• goto	• package	• synchronized
• boolean	• do	• if	• private	• this
• break	• double	• implements	• protected	• throw
• byte	• else	• import	• public	• throws
• case	• enum	• instanceof	• return	• transient
• catch	• extends	• int	• short	• try
• char	• final	• interface	• static	• void
• class	• finally	• long	• strictfp	• volatile
• const	• float	• native	• super	• while

Fig.3. Palabras reservadas.

- **Símbolos**: Se trata de identificadores definidos por el programador. Distinguimos dos clases de símbolos:

  » **Variables**: Representan un espacio de memoria para almacenar un valor de un determinado tipo. El valor de una variable puede cambiar durante la ejecución de un programa. Para utilizar una variable, primero hay que declararla. Esa declaración consiste en asociarle un tipo, enunciar el nombre de la misma y asignar un valor.

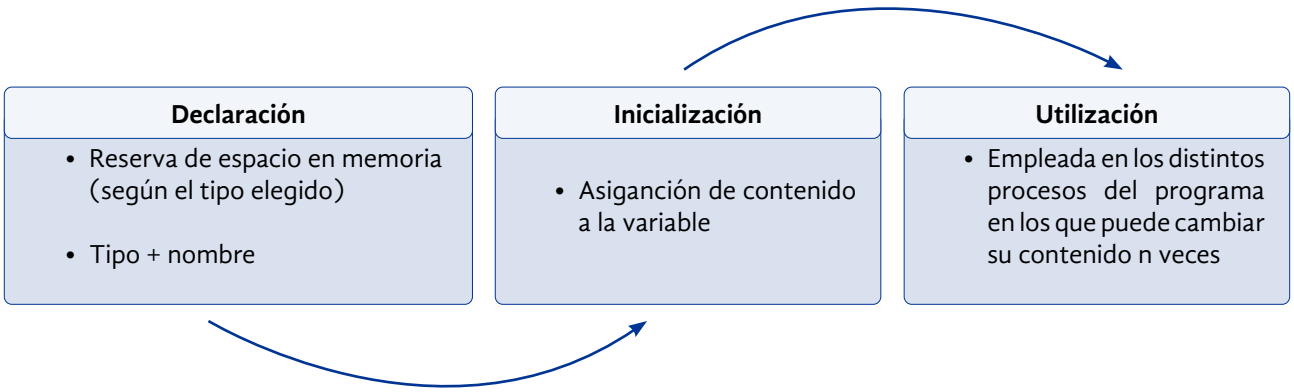
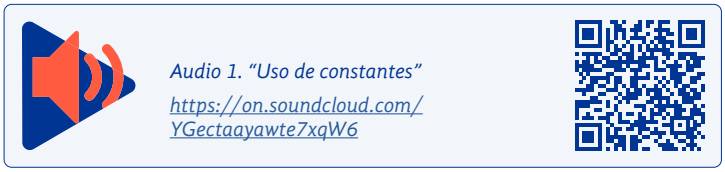


Fig.4. Uso de variables en Java.

  » **Constantes**: Son datos cuyo valor no cambiará a lo largo del programa. Igual que las variables, hay que declararlas, lo que significa indicar el nombre de la constante y asignarle un valor. Para ello, se emplea la palabra reservada '**final**' para declarar constante cualquier tipo de dato.

```
final int valor = 2;
```



Audio 1. "Uso de constantes"
https://on.soundcloud.com/YGectaayawte7xqW6

/ 4. Caso práctico 1: “Variables y constantes”

Planteamiento: Las variables y las constantes son elementos fundamentales en la programación, pero difieren en su naturaleza y uso. Aunque ambas tienen un espacio en memoria para almacenar un valor, las variables cambian el contenido durante la ejecución del programa y, por tanto, se emplean para los datos que se pueden modificar o actualizar a lo largo del tiempo; en cambio, las constantes no cambian durante la ejecución del programa y, por tanto, representan valores fijos que no pueden modificarse.

Fig.5. Constante.

Nudo: Se tienen que declarar las siguientes propiedades:

- Edad: De tipo numérico, se puede incrementar.
- Saludo: De tipo cadena, siempre se dirá ¡Hola!
- Altura: De tipo numérico, se puede modificar.
- Nombre: De tipo cadena, puede emplearse para distintos nombres de personas.
- Número Pi: De tipo numérico, siempre tiene el mismo valor.

¿Qué emplearías para cada una de estas propiedades, variables o constantes?

Desenlace: Como se ha dicho previamente, la principal diferencia entre variables y constantes es el cambio de contenido durante la ejecución del programa, por lo tanto, todas aquellas propiedades que pueden ser susceptibles de cambio serán variables y aquellas que no puedan cambiar serán constantes:

- Edad: Variable.
- Saludo: Constante.
- Altura: Variable.
- Nombre: Variable.
- Número Pi: Constante.

/ 5. Tipos de datos: Primitivos

Cuando necesitamos manejar variables que pueden albergar diferentes tipos de elementos, es importante poder realizar esta tarea con seguridad. Desde el punto de vista de la programación, es fundamental tener en cuenta una de las características más importantes de los datos: su tipo. Es una característica inherente al propio dato que estamos tratando.

Cuando nos centramos en el diseño de programas, el tipo de datos está directamente relacionado con el dominio de valores que pueden almacenar y manejar tanto las variables como las constantes y las propias expresiones que van a conformar nuestro software. Como se puede observar, **el tipo de datos va a determinar el tipo de expresiones que podemos crear a partir de las variables y constantes que estemos utilizando**.

Los tipos de datos más frecuentes utilizados en los diferentes lenguajes de programación son:

- **Tipo de dato simple**: Hace referencia a un tipo que no puede ser divisible, es decir, es un elemento individual del cual no podemos sacar ningún subtipo y comúnmente se denomina **primitivo**. Entre los tipos de datos simples podemos destacar los tipos enteros, coma flotante, lógicos, etc.

- **Tipo de dato estructurado**: Representa un tipo de dato del cual podemos sacar diferentes partes atómicas o simples. Podríamos decir que los tipos de datos estructurados pueden estar formados por tipos de datos simples y/o tipos de datos estructurados.

Haciendo hincapié en los tipos primitivos de datos, su representación va a depender del hardware. De manera general, podemos distinguir tres grandes categorías: **numéricos, lógicos y caracteres**. Existen lenguajes de programación que pueden tener tipos de datos primitivos que estén formados por tipos estructurados. Como podemos observar, el conjunto de datos primitivos disponible en un lenguaje de programación va a depender del propio lenguaje de programación con el que estemos trabajando.

Recuerda...

Los tipos de datos permiten definir el dominio de valores de una variable que es fundamental para poder desarrollar algoritmos correctamente.

En la siguiente figura, se muestran los distintos tipos de datos primitivos.

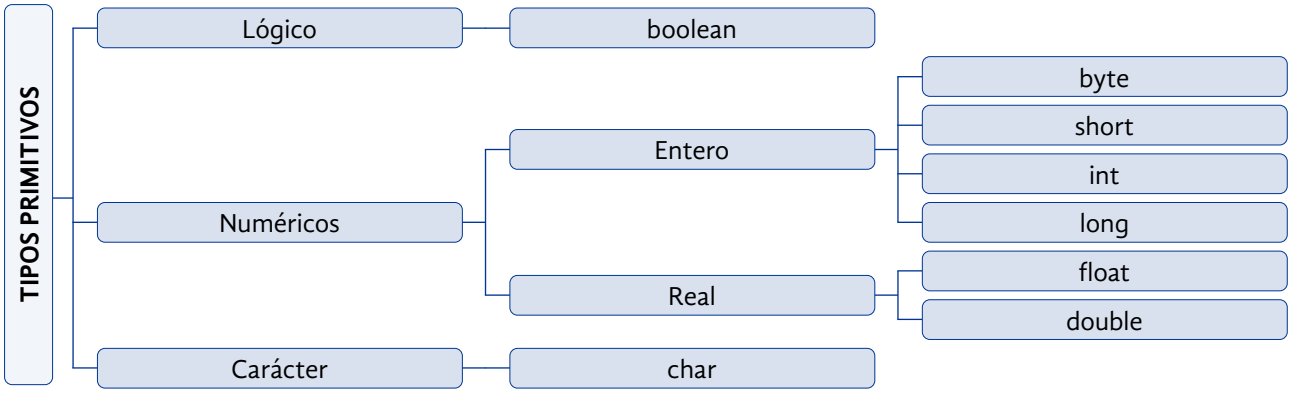


Fig.6. Tipos de datos primitivos.

/ 6. Datos de tipo numérico

Podríamos decir que los tipos numéricos pueden dividirse atendiendo a la **precisión** del tipo de dato con el que estemos trabajando. Normalmente, a mayor precisión en el tipo de dato, mayor número de *bytes* necesitamos para poder almacenar dicho dato. De esta forma, el tipo de datos está directamente relacionado con el espacio de almacenamiento en la máquina.

Por ejemplo, un tipo de dato entero ocupa menos espacio que un tipo de dato real. Una variable de tipo entero tiene menor precisión que una variable definida de tipo real.

6.1. Tipo entero

Los tipos de datos enteros nos permitirán almacenar números enteros tanto positivos como negativos, es decir, números que no tienen una parte decimal, solo una parte entera. En Java, tenemos los siguientes tipos de datos para representar números enteros.

Tipo	Bytes ocupados en memoria	Rango de valores
byte	1	[-128, 127]
short	2	[-32768. 32767]
int	4	[-2^{31}, 2^{31} - 1]
long	8	[-2^{63}, 2^{63} - 1]

Tabla 1. Tipos de datos enteros en Java y su rango.

Por comodidad, se suele emplear el tipo de datos 'int'.

Para poder declarar variables o constantes del tipo entero, lo haremos de la siguiente forma:

```
int nombre_variable;
```

Podremos asignarles un valor directamente en la declaración.

```
int nombre_variable = 8;
```

Si es necesario, podremos declarar más de una variable en la misma línea, separándolas mediante comas.

```
int nombre_variable1, nombre_variable2 = 9;
```

6.2. Tipo real

Los tipos de datos reales nos permitirán almacenar números reales tanto positivos como negativos, es decir, números que tienen una parte decimal además de una entera. En Java, tenemos los siguientes tipos de datos para representar números reales.

Tipo	Bytes ocupados en memoria	Rango de valores	
		En los negativos	En los positivos
float	4	[$-3{,}4E^{38}$, $-1{,}4E^{45}$]	[$1{,}4E^{-45}$, $3{,}4E^{48}$]
double	8	[-1,8E308, -4,9E324]	[$4{,}9E^{-324}$, $1{,}48^{308}$]

Tabla 2. Tipos de datos reales en Java y su rango.

Para poder declarar variables o constantes del tipo real, lo haremos igual que con los enteros, pero, en este caso, por comodidad, siempre vamos a utilizar el tipo *double*.

Vídeo 1. "Declaración de las variables de tiempo real"

https://bit.ly/3cJdTvN

/ 7. Datos de tipo alfanumérico

Nos permitirán almacenar caracteres, es decir, letras, símbolos especiales, números, espacios en blanco, las letras acentuadas tanto en minúscula como en mayúscula, letras de otros alfabetos distintos al latino (como el ruso)...

En Java, tenemos los siguientes tipos de datos para representar datos alfanuméricos:

- **Char**: Representará un único carácter. Se representará entre comillas simples.
- **String**: Representará varios caracteres juntos, es decir, lo que se conoce como una cadena de caracteres. Se representará entre comillas dobles: "Hola, mundo".

Resumiendo, un carácter solo almacena un único elemento, mientras que una cadena puede tener cualquier número de ellos. Cabe destacar que también podemos tener cadenas vacías: "".

Como vemos, tenemos dos posibilidades a la hora de representar un dato alfanumérico, la cuestión es que, si solamente necesitamos representar un único carácter, elegiremos un dato de tipo char, pero si necesitamos representar una o varias palabras, escogeremos un dato de tipo string. Aunque también podremos representar un carácter como un string, siendo una cadena de un único elemento. Recordemos que también podemos tener cadenas vacías: "".

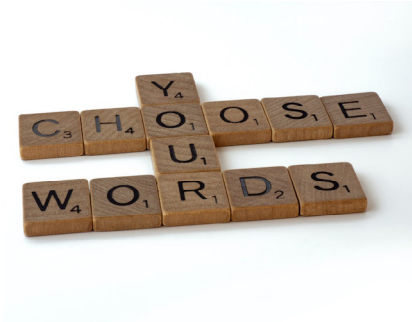


Fig.7. Palabras representadas con cadenas.

Para poder declarar variables o constantes de los tipos char o string, lo haremos de la siguiente forma:

```
char nombre_variable;
String nombre_variable;
```

Podremos asignarles un valor directamente en la declaración.

```
char nombre_variable = 'H';
String nombre_variable = "Hola que tal";
```

Recuerda...

El tipo carácter es diferente al tipo cadena, pues la cadena no puede ser transformada directamente en una representación ASCII. La tabla ASCII es estándar y permite el intercambio de datos con facilidad.

/ 8. Datos de tipo lógico

Los datos de tipo booleano vienen heredados del álgebra de Boole (la que les da su nombre) y pueden representar únicamente dos tipos de valores: verdadero y falso (también conocidos como 0 y 1).

Este tipo de datos no se utilizará para realizar operaciones, sino que su utilidad está en la evaluación de expresiones lógicas para el tratamiento de condiciones y comparaciones. Estas expresiones van a permitir a los programas tomar decisiones a la hora de escoger un camino u otro según si se cumplen unas condiciones u otras.

Con ellos, podremos saber si un número es mayor que otro, si es menor, mayor o igual, menor o igual, o igual. También podremos hacer esas comprobaciones con datos de tipo carácter, tanto de un único carácter como con cadenas de caracteres de cualquier longitud.

Cada vez que necesitemos tomar una decisión, tendremos que realizar comparaciones utilizando expresiones de tipo booleano.

Más adelante, veremos que con este tipo de datos podremos realizar operaciones que nos es imposible ejecutar con los de tipo numérico o carácter.

Fig.8. Camino que sigue un programa.

Para poder declarar variables o constantes del tipo booleano en Java, lo haremos de la siguiente forma:

```
boolean nombre_variable;
```

Podremos asignarles un valor directamente en la declaración:

```
boolean nombre_variable = true;
boolean nombre_variable = false;
```

Si es necesario, podremos declarar más de una variable en la misma línea, separándolas mediante comas:

```
boolean nombre_variable1, nombre_variable2 = false;
```

```
public class Ejemplo {

    public static void main(String[] args) {
        final boolean booleano_constante = true;
        boolean booleano = false;
        boolean booleano1 = true, booleano2 = false;
    }

}
```

Fig.9. Declaración de variables tipo booleano.

/ 9. Expresiones y operadores

Las **expresiones** son combinaciones de constantes, variables, símbolos de operación, paréntesis y nombres de funciones especiales. Cada expresión toma un valor que se determina tomando los valores de las variables y constantes implicadas y la ejecución de las operaciones indicadas. Una expresión consta de "operandos" y "operadores". Dependiendo del tipo de objetos que manipulan, podremos clasificar las expresiones en: aritméticas, lógicas y de carácter.

Los **operadores** son símbolos que indican la operación a realizar. Podemos tener:

- **Operadores aritméticos**: Son los que indican la ejecución de una determinada operación aritmética, como: suma, resta, multiplicación, división, etc.

- **Operadores lógicos**: Son aquellos que indican operaciones relacionadas con el álgebra de Boole. Son operadores lógicos: AND, OR, NOT, NAND, XOR Y NOR.

- **Operadores relacionales**: Son los que señalan la diferencia que hay entre dos operandos. Indicarán igualdad, inferioridad, superioridad, etc.

- **Operador de asignación**: el operador "**=**" se emplea para modificar el valor de una variable. Su sintaxis es la siguiente: variable = expresión; donde a variable se le asigna el valor de la expresión.

9.1. Operadores aritméticos

Los operadores aritméticos son los que nos van a permitir realizar **operaciones aritméticas**. Estos se podrán utilizar con datos de tipo numérico, tanto reales como enteros.

Las operaciones aritméticas que podremos realizar son:

Operación	Símbolo	Descripción
Suma	+	Suma dos números
Resta	-	Resta dos números
Producto	*	Multiplica dos números
División	/	Divide con decimales dos números
Resto de la división	%	Obtiene el resto de una división
Incremento	++	Incremento en 1
Decremento	--	Decremento en 1

Tabla 3. Operaciones aritméticas en Java.

```
public static void main(String[] args) {
    int numero1 = 6, numero2 = 3;

    int suma = numero1 + numero2;
    int resta = numero1 - numero2;
    int producto = numero1 * numero2;
    double division = numero1 / numero2;
    int resto = numero1 % numero2;
}
```

Fig.10. Realización de suma, resta, producto, división y resto en Java.

9.2. Operadores lógicos

Permiten almacenar datos que únicamente pueden tener dos posibles valores: **verdadero o falso**. Aunque parezca un dominio de valores muy pequeño, los tipos de datos lógicos cobran una importancia fundamental a la hora de evaluar expresiones que forman nuestra solución software.

Nos van a permitir realizar operaciones con los datos del tipo booleano. Estos se pueden aplicar únicamente a estos datos. No tiene sentido aplicarlos a un int, double o string.

Estos datos están basados en la lógica matemática y son los siguientes:

- Operador **AND**, representado como &&
- Operador **OR**, representado como ||
- Operador **NOT**, representado como !

Los operadores AND y OR son operadores binarios, esto quiere decir que van a operar sobre dos variables, mientras que el operador NOT es monario, lo que significa que va a operar sobre una única variable.

Su funcionamiento es el siguiente:

- El operador AND resultará verdadero cuando las dos variables sobre las que se aplica sean verdaderas, en caso contrario, devolverá falso.
- El operador OR será verdadero cuando una de las dos variables sobre las que se aplica sea verdadero, en caso contrario, devolverá falso.
- El operador NOT devolverá el valor contrario a la variable a la que se aplica.

Para entender mejor el comportamiento de los operadores lógicos, podemos utilizar una tabla de verdad por cada operación lógica, la cual nos mostrará los posibles resultados de la misma.

AND	TRUE	FALSE
TRUE	TRUE	FALSE
FALSE	FALSE	FALSE

OR	TRUE	FALSE
TRUE	TRUE	TRUE
FALSE	TRUE	FALSE

NOT	
TRUE	FALSE
FALSE	TRUE

Tabla 4. Tablas de verdad de los operadores AND, OR y NOT.

```
public static void main(String[] args) {
    boolean booleano1 = true, booleano2 = false;

    boolean resultado1 = booleano1 && booleano2;
    boolean resultado2 = booleano1 || booleano2;
    boolean resultado3 = !booleano1;
    boolean resultado4 = !booleano2;
}
```

Fig.11. Ejecución de los operadores lógicos en Java.

9.3. Operadores relacionales

Nos van a permitir efectuar operaciones de comparación entre variables del mismo tipo. Estos se pueden aplicar a cualquier tipo de variable salvo a los string.

Es importante recordar que solo se podrán comparar variables con otras del mismo tipo.

Los operadores relacionales disponibles en Java son:

- Operador **igual** que, se representa como ==
- Operador **distinto**, se representa como !=
- Operador **mayor que**, se representa como >

- Operador **menor que**, se representa como <
- Operador **mayor o igual que**, se representa como >=
- Operador **menor o igual que**, se representa como <=

Estos operadores devolverán verdadero o falso, según si se cumple la condición del operador o no.

Cabe destacar también que mediante los operadores lógicos podremos concatenar más de un operador de comparación.

9.4. Operadores opera y asigna

Los operadores de asignación, como su propio nombre indica, asignarán un valor a una variable mediante el operador "=". Sin embargo, existen otros operadores denominados opera y asigna, tal y como refleja la Tabla 5:

Operador	Ejemplo	Equivalencia
=	int a = b;	
+=	int a+= b;	int a = a + b;
-=	int a -= b;	int a = a - b;
*=	int a *= b;	int a = a * b;
/=	int a /= b;	int a = a / b;
%=	int a %= b;	int a = a % b;

Tabla 5. Operadores de asignación.

Estos operandos son los encargados de realizar una operación tomando el primero de ellos el valor a la izquierda de la expresión y como segundo operando el valor a la derecha del "=", asignando el resultado a la variable empleada en el primer operando.

Existe un último operador denominado **operador ternario** que veremos más adelante y cuya expresión es la siguiente:

```
ExpresiónLógica/Relacional ? valor1: valor 2;
```

Este operador devuelve un valor entre dos posibles donde su elección dependerá de una expresión lógica o relacional que puede tomar dos valores. Dependiendo del resultado, seleccionará uno de los dos valores.

/ 10. Caso práctico 2: "Precedencia de operadores"

Planteamiento: En Java, el operador ++ se puede usar tanto antes como después de una variable, pero su ubicación afecta a cuándo se realiza la operación de incremento. Esto se conoce como preincremento (++variable) y postincremento (++variable). La diferencia radica en cuándo se realiza la operación de incremento en relación con la evaluación de la expresión que contiene la variable.

- Con el preincremento, la variable se incrementa antes de que se evalúe la expresión en la que aparece.
- Con el postincremento, la variable se incrementa después de que se evalúe la expresión en la que aparece.

Nudo: Si x es una variable de tipo int con valor 10, determina qué se muestra por pantalla cuando se ejecutan las siguientes instrucciones:

- int x = 10;
- System.out.println(x);
- x++;
- System.out.println(x);
- System.out.println(++x);
- System.out.println(x++);
- System.out.println(x);
- System.out.println(x++);
- System.out.println(++x);
- System.out.println(++x);

Desenlace: A continuación se muestra la ejecución de las distintas instrucciones paso a paso:

Instrucción	Valor x	Pantalla
int x = 10;	10	
System.out.println(x);	10	10
x++;	11	
System.out.println(x);	11	11
System.out.println(++x);	12	12
System.out.println(x++);	13	12
System.out.println(x);	13	13
System.out.println(x++);	14	13
System.out.println(++x);	15	15
System.out.println(++x);	16	16

Tabla 6. Resolución del caso práctico.

/ 11. Conversiones de tipo (casting) y comentarios

Como hemos visto, solo podemos realizar operaciones entre variables del mismo tipo, pero, si nos paramos a pensar un momento, por ejemplo, veremos que se pueden sumar dos números entre sí, los enteros y los reales, al fin y al cabo, son números, aunque son de tipos distintos.

Existen dos tipos de *castings*:

- Implícitos.
- Explícitos.

11.1. Castings implícitos (*Upper-casting*)

Se trata de un tipo de conversión que normalmente no necesita ayuda del programador, pues puede realizarse automáticamente. En ella se intenta traducir un elemento de un tipo menor a un elemento de tipo mayor.

Por ejemplo, cuando pretendemos convertir un número entero en un número real. Desde el punto de vista del hardware, un entero normalmente ocupa 32 bits, mientras que un real ocupa como mínimo 64 bits. Atendiendo al número de bits necesarios para cada tipo de datos, podemos convertir, sin ningún problema, un entero en un real, ya que el tipo de destino es mayor que el tipo origen. En este tipo de conversión no se pierde información.

Recuerda...

La conversión automática de tipos de datos se produce cuando no se pierde información en la conversión de los tipos. La mayoría de los lenguajes permiten la conversión automática, es decir, una conversión de datos implícita.

11.2. Castings explícitos (*Down-casting*)

Este tipo de conversión suele requerir ayuda de programación, pues debe realizarse explícitamente, debido a que se puede perder información durante la conversión. Si volvemos al ejemplo anterior, sería el caso en el que queremos transformar un real en un entero.

En este supuesto, podemos observar que un número real requiere mayor número de bits que un número entero. Con este tipo de conversión estamos perdiendo precisión, por lo que requerimos de la ayuda del programador. En muchas ocasiones, se introducen errores de tipo debido a este tipo de conversión explícita.

Para realizar un casting en Java, tendremos que indicar entre paréntesis el tipo al que queremos convertir el dato.

```
public static void main(String[] args) {
    double numero_real = 8.3;

    int numero_entero = (int) numero_real;
    // Aquí perderemos información, ya que convertimos
    // un double a int.
    // numero_entero valdrá 8, no 8.3
}
```

Fig.12. Casting de double a int en Java.

Los comentarios nos van a servir para explicar qué es lo que hace el código que estamos escribiendo, ya sea para otros programadores o para nosotros mismos pasado un tiempo.

Audio 2. "Comentarios de código"
https://on.soundcloud.com/xqTQoKTkG2TK5bVB9

/ 12. API de Java

Una de las mayores ventajas de emplear un lenguaje de programación moderno es la biblioteca que emplea, de manera transparente, el programador, denominada API.

La API de Java está diseñada para facilitar el desarrollo de aplicaciones, ya que proporciona una serie de funciones y herramientas comunes que los desarrolladores pueden utilizar en sus programas. Esto permite a los programadores centrarse en la lógica específica de sus aplicaciones sin tener que preocuparse por la implementación de detalles de bajo nivel.

12.1. Paquetes

Se trata de un conjunto de clases, interfaces y métodos predefinidos que proporciona Java para el desarrollo de aplicaciones. Está organizado en paquetes que abarcan diferentes áreas funcionales, como entrada/salida, manejo de archivos, *networking*, gráficos, procesamiento de datos, entre otros.

Un ejemplo sería la clase "**Math**" que proporciona las herramientas necesarias para la realización de cálculos matemáticos y se encuentra dentro del paquete "lang", y este a su vez está dentro del paquete "java", por lo el nombre completo para dicho paquete sería java.lang.Math.

12.2. Salida por consola

Una de las operaciones más básicas que realizamos y requiere la API de Java es la salida de mensajes para mostrar información al usuario. Esta información puede ser gráfica o solo texto. En este último caso, se denomina por consola y requiere emplear la clase **System** con los métodos:

```
System.out.print("Mensaje"): Imprime mensaje sin salto de línea
```

```
System.out.println("Mensaje"): Imprime mensaje con salto de línea
```

12.3. Entrada de datos

En el caso de la entrada de datos, es necesario emplear la clase Buffer o la clase Scanner. Si empleamos la clase Scanner debemos seguir siempre los mismos pasos:

- Crear un nuevo escáner:

```
Scanner lectura = new Scanner(System.in);
```

- Indicar qué vamos a leer por el teclado

lectura.nextInt(): lee un número entero

lectura.nextDouble(): lee un número doble

lectura.nextLine(): lee una cadena de caracteres hasta un salto de línea

lectura.next(): lee una cadena de caracteres hasta encontrar un espacio en blanco

Video 2. "Lectura de datos por teclado"
https://bit.ly/30ikVVS

/ 13. Resumen y resolución del caso práctico de la unidad

En este tema se han visto los distintos tipos de datos, variables y constantes y cómo les afectan los bloques de código.

También se han expuesto las distintas operaciones que se pueden realizar con cada tipo de dato. Hemos visto que no serán las mismas para unos que para otros y que es posible utilizar castings entre tipos para facilitarnos las cosas.

Por último, se han introducido los comentarios de código, explicando qué son y qué utilidad tienen.

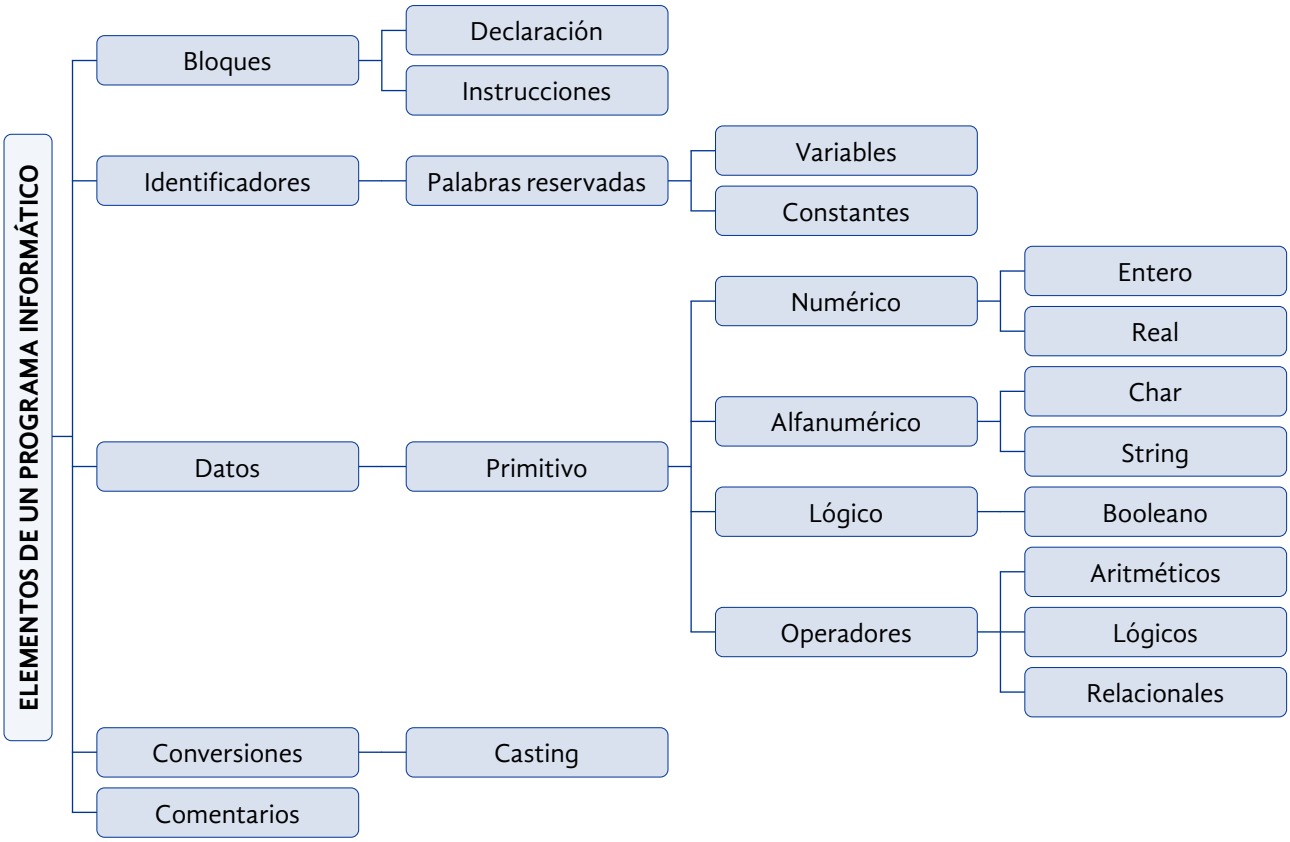


Fig.13. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Se puede determinar si un número es par o impar en Java utilizando la operación módulo (%). Si un número es divisible por 2 con un resto = 0, entonces es par; de lo contrario, es impar.

El pseudocódigo sería el siguiente:

```
Inicio
        Declaramos número
        Si número resto 2 = 0
        El número es par
        Fin sí
Si no
El número es impar
        Fin si no
Fin
```

/ 14. Bibliografía

Wikipedia contributors (s. f.). *Bloque de código*. Wikipedia, The Free Encyclopedia. Recuperado de: https://es.wikipedia.org/w/index.php?title=Bloque_de_c%C3%B3digo&oldid=148638778

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ramírez, J. M. (2019, 23 de marzo). Código Fuente. Identificadores, variables y constantes. másQteclas. Recuperado de: https://www.masqueteclas.com/masqueteclas/013-identificadores-variables-y-constantes/